

WASP: Selective Data Prefetching with Monitoring Runtime Warp Progress on GPUs

Yunho Oh, *Student Member, IEEE*,
 Myung Kuk Yoon, *Student Member, IEEE*,
 Jong Hyun Park, *Student Member, IEEE*,
 Yongjun Park, *Member, IEEE*, and
 Won Woo Ro , *Senior Member, IEEE*

Abstract—This paper proposes a new data prefetching technique for Graphics Processing Units (GPUs) called Warp Aware Selective Prefetching (WASP). The main idea of WASP is to dynamically select warps whose progress is slower than that of the current warp as prefetching target warps. Under the in-order instruction execution model of GPUs, these prefetching target warps will certainly execute the same load as the current warp. Exploiting that, WASP prefetches the data for prefetching target warps, which allows the prefetched data to be accurately accessed. To simply verify the progress of the warps, WASP monitors the counts of the dynamic load executions for all warps. When a warp executes a load, WASP searches the warps with lower load execution counts than the current warp and generates the prefetch requests for them. In our evaluation, WASP achieves a 16.8 percent speedup compared to the baseline GPU.

Index Terms—GPGPU, data prefetching, warp scheduling, cache performance

1 INTRODUCTION

HIGH-THROUGHPUT Graphics Processing Units (GPUs) have become important processing engines in energy-efficient heterogeneous computing systems [1]. However, GPUs often suffer from performance degradation with memory-intensive workloads. One of the main reasons is the limit on memory system performance. The off-chip memory latency takes hundreds of cycles. Thus, the stalls increase if the memory requests are congested over the limit of the memory subsystem resources in a small time window [2], [3], [4]. Even though some memory latencies can be hidden by the interleaved thread executions, a large number of stalls are still created. In our initial analysis, the stall cycles caused by long latency memory operations consume 68.8 percent of the total execution time in memory-intensive applications.

Cache memory structures can help to reduce the average memory latency. However, the cache memory is not efficiently utilized on GPUs for the following reasons. First, the capacity of the data cache is not sufficient for hundreds of threads [5], [6], [7]. Cache lines fetched from the threads are frequently contended in the small-sized cache. Second, a large portion of cache lines are fetched from streaming data, which are hardly reused [8]. For these reasons, GPUs still suffer from a high cache miss ratio.

GPU applications often show regular memory access patterns [9]. By exploiting them, data prefetching can improve the GPU performance by efficiently hiding the memory latency. One of the important design issues for prefetching is the prefetching timeliness [10],

[11]. The ideal prefetching on GPUs schedules threads that will access the data just prefetched in the cache. Despite the regular memory access patterns on GPUs, it is a challenge to prefetch the necessary data with such ideal timing. On GPUs, multiple threads are grouped into a warp, and the warps are scheduled in an out-of-order fashion [12]. Such a unique warp scheduling mechanism on GPUs makes it difficult to predict the memory addresses that will be accessed in the future. In this situation, the prefetcher may request the cache lines after the warps have already accessed them. These redundant prefetches do not improve the performance. The cache lines fetched by redundant prefetches are rarely accessed by the threads. Moreover, they may evict other cache lines that will be re-accessed shortly. In our experiments with the previously proposed prefetching techniques for GPUs [13], [14], 24.7–26.0 percent of the total number of prefetches were redundant.

To reduce the number of redundant prefetches and provide useful prefetches, we propose a new hardware data prefetching architecture called Warp Aware Selective Prefetching (WASP). The main idea of WASP is to select the warps that will execute load instructions in the near future at runtime and prefetch the data for them. We call the selected warps as prefetching target warps. To properly select prefetching target warps, we consider a unique behavioral feature of the Single-Instruction Multiple Threads (SMT) execution model on GPUs, which is that all warps in a GPU kernel execute the same instruction stream in an in-order fashion [15]. This implies that if a load is executed by warp *A*, the same load will certainly be executed by other warps whose progress is slower than warp *A*. Exploiting this characteristic, WASP selects warps with slower progress than the warp currently executing a load as prefetching target warps and generates the prefetch requests for them.

To monitor the progress of a warp, the total number of instruction executions of a warp can be used as a key metric. However, monitoring instruction execution counts for tens of active warps requires a large overhead. Instead, WASP monitors only the dynamic load execution count for each warp. With this scheme, when a warp executes a load, WASP selects warps with load execution counts that are smaller than that of the current warp as prefetching target warps. In addition, we observe that while the warps execute the load instructions in an arbitrary order, a large portion of the strides between two neighbor warps are equal to the previously calculated strides. With this observation, WASP employs an inter-warp stride prefetching scheme.

WASP has the following flow. When a warp executes a load, the load execution count of the current warp is incremented. In addition, WASP compares the load execution counts of other warps with that of the current warp. The warps with smaller load execution counts are considered to be prefetching target warps. At the same time, an inter-warp stride is calculated. If this stride is the same as the previously calculated stride, it is considered to be a regular stride, and WASP uses it to generate the prefetch requests for prefetching target warps.

In our simulation, WASP reduces the ratio of redundant prefetches to less than 10 percent. As a result, WASP achieves a 16.8 percent speedup compared to the baseline GPU.

2 RELATED WORK

Several prefetching techniques have been introduced for a many-core GPU architecture. Lee et al. proposed a Many-Thread Aware (MTA) prefetching architecture for GPUs [13]. MTA includes two techniques: per-warp stride prefetching and inter-thread prefetching. Sethia et al. introduced an intra-warp stride prefetching technique called Adaptive Prefetching on GPUs for Energy Efficiency (APOGEE) [8]. APOGEE focused on improving the energy efficiency by reducing the number of active threads and the size of the

- Y. Oh, M.K. Yoon, J.H. Park, and W.W. Ro are with the School of Electrical and Electronic Engineering, Yonsei University, Seoul, Korea. E-mail: {yunho.oh, myungkuk.yoon, park.jonghyun, wro}@yonsei.ac.kr.
- Y. Park is with the Division of Computer Science and Engineering, Hanyang University, Seoul, Korea. E-mail: yongjunpark@hanyang.ac.kr.

Manuscript received 13 July 2017; revised 27 Dec. 2017; accepted 5 Jan. 2018. Date of publication 7 Mar. 2018; date of current version 15 Aug. 2018.

(Corresponding author: Won Woo Ro.)

Recommended for acceptance by J. D. Bruguera.

For information on obtaining reprints of this article, please send e-mail to: reprints@ieee.org, and reference the Digital Object Identifier below.

Digital Object Identifier no. 10.1109/TC.2018.2813379

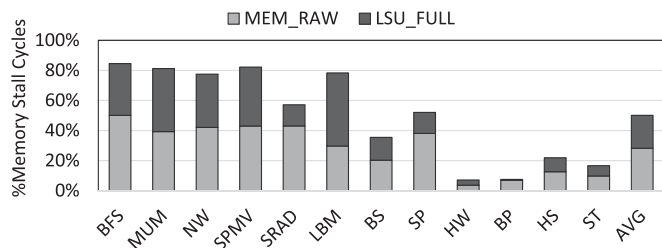


Fig. 1. Portion of stall cycles due to memory system.

register file. Jog et al. introduced a harmonic mechanism for warp scheduling that exploits memory bank-level parallelism and pre-fetching [14]. Lakshminarayana et al. proposed a new way to pre-fetch data on the spare registers of GPUs [16]. They proposed a specialized prefetching mechanism for graph algorithms.

Several studies have worked toward efficient cache utilization on GPUs. Li et al. analyzed the performance impact of the shared memory and data cache on GPUs [17]. Li et al. proposed a novel cache allocation technique to reduce the cache contentions on GPUs [18]. Lee et al. introduced a combination of a new warp scheduling technique that accelerates executions of slowest warp and a cache prioritization technique [19].

Our work focuses on improving the performance of a GPU using a hardware prefetching scheme. We mainly have the goal of improving the prefetching efficiency by providing timely pre-fetches considering the behavioral features of GPUs.

3 MOTIVATION

3.1 Limit of Memory System on GPUs

The memory system used on modern GPUs causes two types of stalls. First, an instruction in a warp cannot be executed if a preceding long latency memory instruction is waiting for its result. We call the resulting stalls a Read-After-Write (RAW) dependency on MEM_RAW. Second, if the memory instructions are congested in a small time window, some instructions cannot be executed because the Load Store Units (LSUs) are full. The stalls caused by congestion on the LSUs are called LSU_FULL in this paper. We measure the portion of MEM_RAW and LSU_FULL cycles out of the total number of execution cycles.

Fig. 1 shows the portion of MEM_RAW and LSU_FULL cycles out of the total number of execution cycles for GPU applications. The details of these applications are provided in Section 5.2. In the graph shown in this figure, the eight applications on the left show aggregated MEM_RAW and LSU_FULL cycles of more than 30 percent. The average ratio of MEM_RAW and LSU_FULL cycles in these applications is 68.8 percent. We classify these applications as memory-intensive ones. On the other hand, the remaining four applications show less than 25 percent of the MEM_RAW and LSU_FULL cycles. For these applications, the limit of the memory system performance is not critical to the overall performance, while the GPU performs a large portion of arithmetic operations. Therefore, we consider them to be compute-intensive applications.

As shown in these experimental results, congestion in long latency memory operations causes a large percentage of stalls in memory-intensive applications. Even though GPUs provide a wide memory bandwidth, the long memory latency increases the waiting time for the numerous memory requests from thousands of threads. Because of these stalls in the memory system, GPUs often suffer from performance degradation.

3.2 Challenge of Prefetching on GPUs

GPU applications often show regular memory access patterns [7], [8], [13]. If the data that will be used in the future is properly pre-fetched by exploiting the regular access patterns, the average memory latency can be reduced.

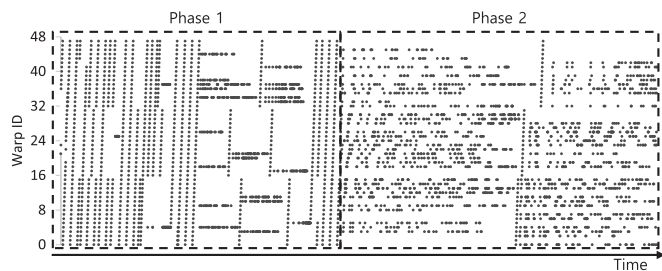


Fig. 2. Example of load execution order on an SM.

One of the most important aspects when implementing the pre-fetching technique in GPUs is the SIMT execution model. All of the threads on GPUs execute the same instruction stream in an in-order fashion. On the other hand, the active threads on a GPU are not scheduled in a predetermined order.

The scheduling order of threads may become arbitrary for the following reasons. First, the thread scheduling order depends on the scheduling policy. On GPUs, threads are grouped into a warp, and the warps are scheduled by the warp schedulers on Streaming Multiprocessors (SMs). The warp schedulers search the available warps with their scheduling policies and try to issue one of them in every cycle. The most basic policy is loose round-robin (LRR). However, many previous studies have argued that the LRR policy causes the hardware resource underutilization problem. To solve this problem, several policies that schedule the warps with different orders have been proposed [3], [12], [20]. Second, not all of the warps have the same progress because of the dynamic execution status. For example, the cache size is not sufficient for tens of warps. Therefore, the warps have different cache access results. The warps with frequent cache hits can quickly proceed to the next instruction. Thus, at an instance, those warps can show better progress than other warps that frequently miss the cache. In this situation, when a warp scheduler searches the available warps, the selected warp may not be the sequentially next warp (i.e., the warp whose ID is the current warp ID + 1).

Fig. 2 shows an example of the dynamic warp scheduling status. It shows the order of load executions on an SM while the baseline GPU executes a Breadth First Search (BFS) application within a time window. The X axis represents the execution time, and the Y axis represents the IDs of active warps (a total of 48 warps). Each spot in this graph represents the ID of a warp that executes a load instruction at an instance. The graph can be divided into two phases. In most of Phase 1, the warps execute the loads in sequential order of warp IDs. On the other hand, the load execution order becomes arbitrary in Phase 2. This execution feature makes it difficult to predict the memory addresses that will be accessed in the future.

3.3 Redundant Prefetching Analysis

Under the situation of unordered load executions across warps, as shown in Fig. 2, the data prefetching techniques may prefetch the data after warps access them. We call these prefetches redundant prefetches. Redundant prefetches are harmful to the performance for the following reasons. First, they may evict the useful lines that will be accessed in the near future from the L1 cache. This increases the cost to fetch the useful lines again from the shared L2 cache or the off-chip device memory. Second, they spend memory system resources, including the Miss Status Holding Registers (MSHRs) and request queue in the off-chip memory. They also increase the queuing delay of the other memory requests.

We analyzed how many redundant prefetches occurred while executing GPU applications. We used two previously introduced prefetching schemes for GPUs in this analysis. First, we used the stride-based prefetching scheme, called MTA [13]. MTA calculates the per-warp and per-program counter (PC) based strides and

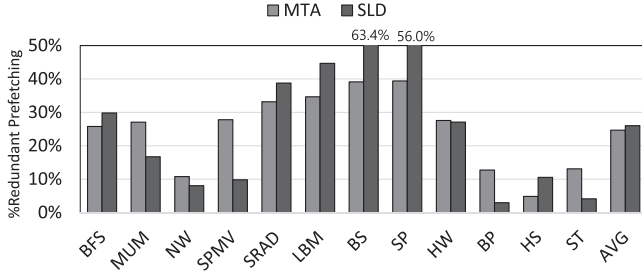


Fig. 3. Ratio of redundant prefetches on GPUs.

prefetches the data for the sequentially next warp. Second, we used spatial locality detection-based prefetching (SLD) [14]. This scheme considers four consecutive cache lines to be a macro block. If two lines in a macro block are fetched by the loads, the remaining two lines are prefetched with this scheme.

Fig. 3 shows the ratio of redundant prefetches, which are calculated by dividing the number of prefetched cache lines that have already been accessed with the loads by the total number of prefetched cache lines. The MTA and SLD prefetchers show average redundant prefetching ratios of 24.7 and 26.0 percent, respectively. In particular, they generate large numbers of redundant prefetches in SRAD, LBM, BS, and SP. These applications show regular memory access patterns compared to the remaining memory-intensive applications (details will be provided later). However, as mentioned in Section 3.2, the load execution across warps become unordered as warps execute more instructions. This feature makes it difficult for the prefetchers to accurately predict the future memory addresses even though they find the memory access regularity.

4 WASP ARCHITECTURE

4.1 Main Idea

To provide efficient prefetching on GPUs, we observed that it is important to find the warps that will execute the loads. To reduce the redundant prefetches and provide more accurate prefetches, we design a new data prefetching architecture, called WASP.

WASP exploits a key feature of the SIMT execution model on GPUs. After Cooperative Threads Arrays (CTAs) are launched to run an application, all of the warps execute the same instruction stream in an in-order fashion. In this execution model, when a warp executes a load instruction, it is obvious that the warps with slower progress than the current warp will execute the same load in the future.

Fig. 4 depicts an execution example with three warps. All three warps execute 12 instructions. In this example, Warp 1 currently executes the 8th instruction, while Warp 0 and Warp 2 are executing the 7th and 12th instructions, respectively. We can find that Warp 0, whose progress is slower than that of Warp 1, will certainly execute the 8th instruction in this example.

By exploiting this feature of the SIMT execution model, WASP selects the warps whose progress is slower than that of the currently issued warp and prefetches the data for them. We call the selected warps prefetching target warps. The proposed scheme provides prefetches only for the future load executions. Therefore, it reduces the redundant prefetches. Considering that the warps execute the same instruction stream, the prefetching target warps can be verified by comparing the dynamic instruction execution counts between warps. In the example of Fig. 4, the load execution counts of Warp 0, Warp 1, and Warp 2 are 7, 8, and 12, respectively. By comparing these counts, we can also find that Warp 0 has slower progress than Warp 2.

However, counting the dynamic instruction executions for all the warps requires a large overhead if each warp executes millions of instructions. To verify the progress of each warp with a small overhead, WASP monitors the global load execution counts of the

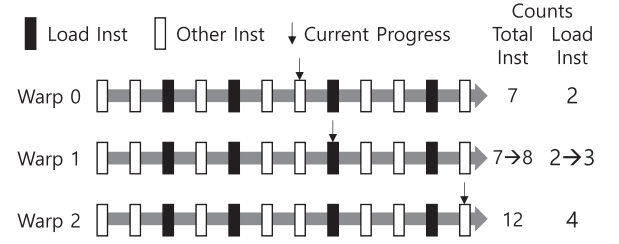


Fig. 4. Relation between warp progress and load execution count (Assuming warp 1 currently executes the 8th instruction).

active warps, instead of the total instruction execution counts. In the above example, when Warp 1 executes the 8th instruction, which is the third load instruction, the load execution counts of Warp 0 and Warp 2 are 2 and 4, respectively. Just as with the scheme for the dynamic instruction count comparison, we can find that warp 0 will execute the same third load in the future. With this observation, WASP selects the warps whose load execution counts are less than that of the currently issued warp as prefetching target warps.

To support prefetching for prefetching target warps, WASP employs the inter-warp stride prefetching scheme. WASP calculates the per-warp and per-PC stride, which is the memory address delta between the accesses of two neighbor warps executing the same load. As mentioned in Section 3.2, the warps are scheduled in an arbitrary order. Thus, the address delta between two recent memory accesses cannot be the per-warp stride. Therefore, WASP calculates the inter-warp stride with the two latest memory requests from different warps generated by the same static load as follows.

$$\text{inter_warp_stride} = \frac{\text{Addr2} - \text{Addr1}}{\text{WID2} - \text{WID1}} \quad (1)$$

Addr1 and Addr2 are the memory addresses requested from the same static load. WID1 and WID2 are the IDs of the warps that request Addr1 and Addr2, respectively.

We observe that regular inter-warp strides are frequently detected in GPU applications even though the warps execute the loads in an arbitrary order. A regular inter-warp stride is verified if the currently calculated stride is equal to the previously calculated stride for the same static load. We measure the ratio of regular inter-warp strides, which is calculated by dividing the number of regular inter-warp strides by the total number of inter-warp strides. Fig. 5 shows the experimental results. On average, 50.0 percent of the total inter-warp strides are regular. SRAD, LBM, BS, and SP generally show regular memory access patterns. Thus, the regular inter-warp stride ratios become greater than 70 percent. On the other hand, the memory access patterns are relatively irregular in BFS, MUM, NW, and SPMV. Thus, the regular inter-warp stride ratios are lower than those for applications with regular patterns. In the case of compute-intensive applications (HW, BP, HS, and ST), the ratios vary greatly with the application. With these results, we find that inter-warp stride prefetching can properly predict the memory addresses that are soon to be accessed.

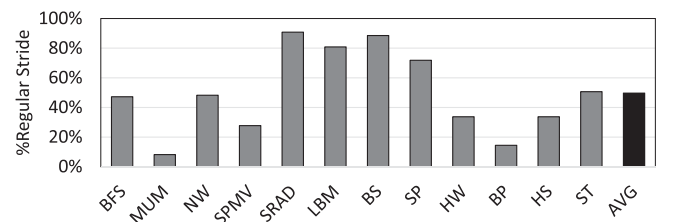


Fig. 5. Portion of regular inter-warp stride.

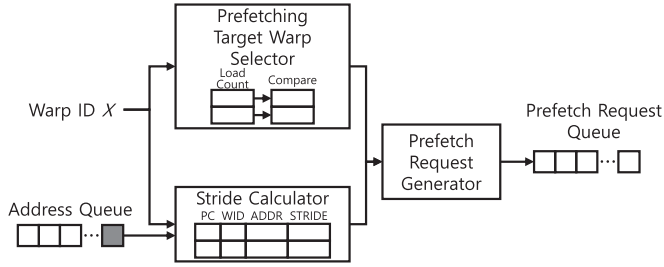


Fig. 6. Architectural overview of WASP.

4.2 Architectural Overview

Fig. 6 depicts an architectural overview of WASP. When a warp executes a load, the currently requested memory addresses are stored in the address queue. Then, the prefetching target warp selector compares the load execution counts between the current warp and the other warps. The IDs of the selected prefetching target warps are sent to the prefetch request generator. At the same time, an inter-warp stride is calculated by the stride calculator. The stride calculator stores the inter-warp stride for each static load. This module determines whether the currently calculated stride is regular by comparing it with the previously calculated stride. If the stride is regular, the prefetch request generator calculates the prefetching addresses for the target warps and generates the prefetch requests for them. Prefetch requests are processed after the memory requests generated by the current load. All of the values in the address queue and prefetch request queues are kept until a new load is executed. If a new load is executed, they are flushed.

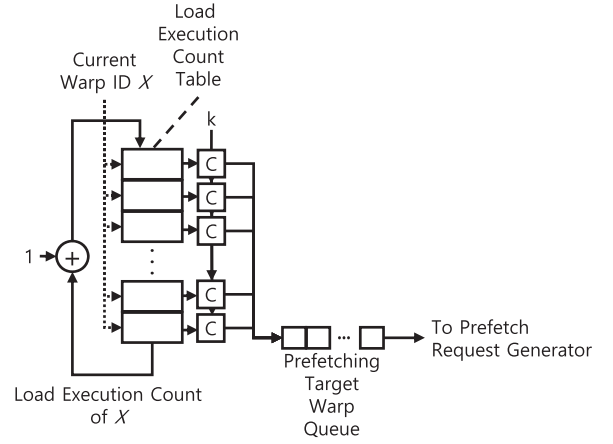
4.3 Detailed Implementation

4.3.1 Prefetching Target Warp Selector

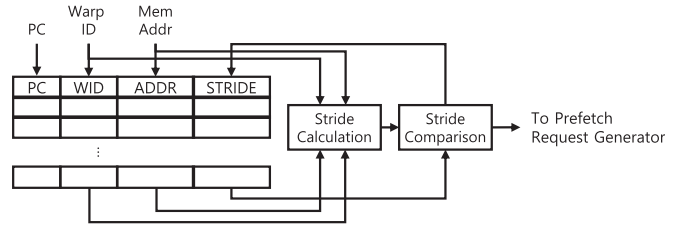
Fig. 7a shows the detailed structure of the prefetching target warp selector. This module performs two functions when a warp executes a load. One is to increment the load execution count of the current warp, and the other is to find prefetching target warps. These functions are performed using a simple structure, called the load execution count table. The load execution count table has as many entries as the maximum number of active warps (48 in this paper). Each entry stores the load execution count of a warp and is indexed by the warp ID.

The load execution count of the current warp is incremented by simply adding one to the previously stored load execution count in the entry for the current warp. For the prefetching target warp searching function, as many comparators are allocated as the number of load execution count table entries. They compare the load execution counts between the current warp and the warps in the remaining entries. When a warp executes N th loads, the comparator for each warp calculates $\text{load execution count} - (N+k)$. k in Fig. 7a is a variable that is used to adjust the prefetching distance (details will be provided later). If the computation result for a warp is less than zero, the corresponding warp becomes a prefetching target warp. The IDs of the selected prefetching target warps are stored in the prefetching target warp queue. In the prefetching target warp queue, the IDs that are greater than the current warp ID are first stored in ascending order. Then, the remaining warps are stored.

Dynamic Prefetching Distance Adjustment. The prefetching effect can be adjusted by dynamically controlling the prefetching distance [21]. For that purpose, WASP introduces a runtime prefetching distance adjustment scheme. As previously mentioned, WASP determines the prefetching target warps by comparing the result of the $\text{load execution count} - (N+k)$, where k is the offset of load executions between the current warp and the remaining warps. By dynamically adjusting k , the prefetching distance can be controlled. At the beginning of a GPU kernel, this value is initially set to the minimum value (1) and dynamically increased or decreased



(a) Prefetching target warp selector (C: Comparator)



(b) Stride calculator

Fig. 7. Main modules in WASP.

according to the results of prefetches. If a redundant prefetch is detected, k is incremented by one to increase the prefetching distance. When a prefetch request hits the L1 cache or merges in an MSHR, it is considered to be a redundant prefetch. On the other hand, if an early eviction of a prefetched cache line is detected, k is decreased by one. Each cache line has a bit field to detect early evictions. If a cache line has recently been accessed by a regular memory request, the bit in the corresponding line is set to false. If a cache line is fetched by a prefetch request, the bit for that line is set to true. When a regular memory request misses the L1 cache, an early eviction is detected if the bit of the currently selected victim line is true. WASP finds the optimal prefetching distance at runtime using this scheme.

4.3.2 Stride Calculator and Prefetch Request Generator

Fig. 7b shows the structure of the stride calculator. It consists of a stride table, stride calculation module, and stride comparison module. If a global load is executed, the stride calculator updates the warp ID and requested memory address in the stride table entry that has the same PC address. If there is no stride table entry with the PC address of the current load, this information is updated in a new entry. Whenever an entry is updated, an inter-warp stride is calculated using Equation (1). Then, the stride calculator verifies whether the calculated stride is regular or not. If the current stride is equal to the previously calculated stride in the stride table entry with the current load PC, this stride is considered to be regular. The stride is stored into the STRIDE field whenever a stride is calculated.

If a regular stride is detected, the prefetch request generator generates the prefetch request for the following prefetching address.

$$PAddr = BA + (PIWID - CWID) \times Stride \quad (2)$$

Here, $PAddr$ is the prefetching address, and BA is the base address demanded by the current load. $PIWID$ and $CWID$ are the IDs of a prefetching target warp and the current warp, respectively.

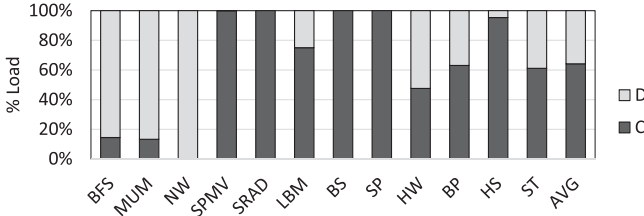


Fig. 8. Portion of loads in converged control flow and diverged branches (C: Converged control flow, D: Diverged branch).

Stride Calculation in Memory Divergence. If a load generates more than one memory request, multiple inter-warp strides can be calculated. This requires an excessive computation overhead. Moreover, the inter-warp stride may often become irregular unnecessarily. To prevent this confusion, the stride calculator stores the memory address that is first requested in a load and calculates an inter-warp stride using it.

4.3.3 Optimization of WASP

To minimize the negative effects of prefetching, WASP includes the following simple optimizations.

Regulating Prefetch Request Generation. If the prefetch requests are too congested, the regular memory requests may be delayed. These executions increase the stalls in the memory pipeline, and such an aggressive prefetching may result in performance degradation. To minimize this effect, WASP generates the prefetch requests only if there are available MSHRs. If the MSHRs are full, the prefetch request generator does not generate the prefetch requests even though a regular stride is found.

Disabling Prefetching for Loads in Diverged Branches. To avoid any problem that can be caused by diverged branches, WASP considers loads in the converged path only for prefetching candidates. To count the load executions only in the converged path, WASP monitors the SIMT stack, which is the structure used to keep the control flow information of each warp [22]. Each stack entry has a per-warp 32-bit vector called an active mask. The active mask indicates the valid threads in a control flow by setting the corresponding bits to true. When a load is executed, WASP sees whether the SIMT stack for the warp has a single entry and whether all of the bits of the active mask on the SIMT stack entry are true. If so, the load is considered to be in a converged path.

We investigated the portion of loads in diverged branches among the total loads. Fig. 8 shows the analysis results. In this figure, only three applications with irregular memory access patterns (BFS, MUM, and NW) show large percentages of the loads in the diverged control flow. The remaining nine applications show more than 60 percent of the loads in the converged control flow. If the proposed optimization is applied, WASP does not perform prefetching frequently in BFS, MUM, and NW. In fact, these applications do not show performance improvements with prefetching because of the irregular memory access patterns.

4.4 Hardware Overhead

In the prefetching target warp selector, the load execution count table consists of as many 4-byte elements as the maximum number of active warps (48). Therefore, it requires 192 bytes. Each entry of the load execution count table is connected to a comparator with a 32-bit subtractor and 1-byte element to store the warp ID. We observed that 48 subtractors required less than 0.2 percent of an SM's area [23]. The prefetching target warp selector also has a prefetching target warp queue with 32 1-byte entries. To detect the early evictions, each cache line has a bit. Thus, a total of 256 bits are added to the 32 KB L1 cache. In the stride calculator, the stride table contains entries that are full enough to store the information for all the loads in a kernel. In the applications used in this study,

TABLE 1
Simulation Configuration

GPU Core	15 SMs, 1400 MHz, Maximum 48 active warps
Warp Scheduler	Greedy-Then-Oldest (GTO), Dual schedulers per SM
L1 Data Cache	8-way, 32 KB, 128B line, 96 MSHRs
L2 Shared Cache	8-way, 768 KB, 128B line, 96 MSHRs
Off-chip	924 MHz, 6-partitioned, FR-FCFS,
Device Memory	128 requests/partition, BW: 173.25 GB/s
Branch Control	Immediate post dominator

we observed that 16 stride table entries could cover all of the loads for each kernel. Each entry requires a 32-bit PC, 6-bit warp ID, 48-bit address, and 48-bit stride field (for a total of 16.75 bytes). Therefore, the stride calculator consumes 268 bytes. The prefetch request queue consists of 32 48-bit elements. In total, WASP requires 764 bytes of storage overhead and a handful logic per SM.

5 EVALUATION

5.1 Methodology

We used GPGPU-Sim v3.2.2 [26] and the baseline GPU configured as described in Table 1. We compared the performance of WASP with those of three previously introduced prefetching techniques for GPUs. We used MTA [13] and SLD [14], which were also used for the initial analysis reported in Section 3. Moreover, we modeled one more prefetching scheme, called APOGEE [8]. APOGEE detects the per-warp strides, similar to MTA. However, it prefetches the data for the next iterations of the currently issued warp in a way that is different from MTA.

5.2 Benchmarks

Table 2 lists the GPU applications that we used in this paper. We selected 12 applications from Rodinia, [24], Parboil [25] benchmark suites, and CUDA SDK [27]. The table also lists the number of dynamic instructions, input dataset, and average L1 cache accesses per 1000 instructions of each application. We compiled the applications with NVCC and -O2 flag. We also used the PTXPlus feature to simulate realistic workloads. We classified the types and memory access patterns of the applications according to the analysis results of Figs. 1 and 5.

5.3 Effect of WASP Optimizations

Prior to comparing the performance of WASP to those of the previously proposed techniques, we compare the Instructions Per Cycle (IPC) of WASP according to the optimization techniques explained in Section 4.3.3. Fig. 9 shows the experimental results. In this figure, WASP+M includes only the technique that prevents the generation of the prefetch requests when MSHRs are full. WASP+MB includes

TABLE 2
GPGPU Applications for Benchmark (APKI: L1 Cache Accesses Per Kilo Instructions, M: Memory-Intensive, C: Compute-Intensive, R: Regular Memory Access Pattern, I: Irregular Memory Access Pattern)

Application	Input	#Inst	APKI	Type	Pattern
Breadth-First Search (BFS) [24]	1M nodes	438M	33.4	M	I
MUMmerGPU (MUM) [24]	5M ref, 1M queries	40.5B	21.1	M	I
Needleman-Wunsch (NW) [24]	4K×4K nodes	795M	15.5	M	I
Sparse-Mat. Vec. Mul. (SPMV) [25]	146689×146689	1.3B	10.1	M	I
SRAD [24]	4K×4K nodes	9.2B	37.5	M	R
Lattice-Boltzmann (LBM) [25]	120×120×150	52.1B	61.5	M	R
BlackScholes (BS) [15]	4M elements	371M	8.0	M	R
Scalar Product (SP) [15]	4K×4K	372M	13.5	M	R
HeartWall (HW) [24]	48.4 MB AVI	32.8B	7.9	C	I
BackPropagation (BP) [24]	1M nodes	2.83B	3.5	C	I
Hotspot (HS) [24]	1K×1K nodes	420M	1.0	C	I
Stencil (ST) [25]	512×512×64×100	85.2B	1.9	C	R

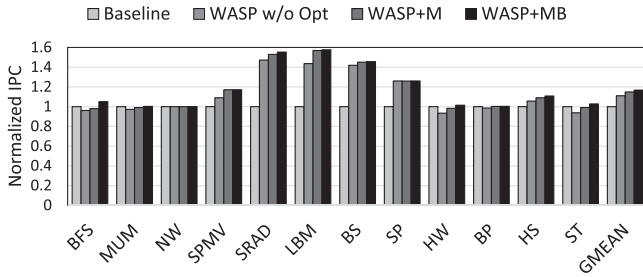


Fig. 9. Performance effect of WASP optimizations (M: Preventing prefetching when MSHRs are full, B: Disabling prefetching for loads in diverged branches).

the technique that disables prefetching for loads in diverged branches, in addition to WASP+M. WASP without any optimization shows a 12.3 percent performance improvement compared to the baseline GPU. WASP+M mitigates the congestion of memory requests in SRAD, LBM, BS, and HS. Therefore, the performance is improved in these applications. On average, WASP+M shows a 3.9 percent better speedup than WASP without optimizations. In the case of WASP+MB, it prevents the slowdown in BFS and MUM, which are memory-intensive and have irregular memory access patterns, by preventing harmful prefetches. As a result, WASP+MB shows a more improved performance, with a 16.8 percent speedup on average. Based on these results, we observe that the optimization techniques contribute to the performance improvement by effectively reducing the harmful prefetches.

5.4 Performance Comparison with Previous Approaches

Fig. 10 shows the performance of SLD, APOGEE, MTA, and WASP. Overall, WASP outperforms the previously proposed techniques. SLD and APOGEE show relatively lower performance than MTA and WASP. Their average speedup values are 4.3 percent (SLD) and 7.4 percent (APOGEE). MTA shows better performance than SLD and APOGEE. Its average speedup was 9.9 percent in our evaluation.

Not all of the prefetching techniques show significant performance improvements in BFS, MUM, NW, and SPMV. SLD, APOGEE, and MTA generate large numbers of redundant prefetches, as analyzed in Section 3.3. As a result, they even show performance degradations in some of these applications compared to the baseline.

On the other hand, WASP does not show such a slowdown. This is because WASP prevents the generation of potentially harmful prefetch requests, while executing the loads in the diverged branches. Moreover, in these applications, the inter-warp strides are often irregular. Therefore, WASP generates fewer prefetch requests than the other techniques. In contrast to the other techniques, WASP achieves a performance improvement of more than 10 percent in SPMV. As analyzed in Section 4.3, all of the loads in SPMV are in the converged control flow. Even though the portion of regular inter-warp strides is approximately 30 percent in this application, as shown in Fig. 5, WASP properly utilizes them and achieves the performance improvement.

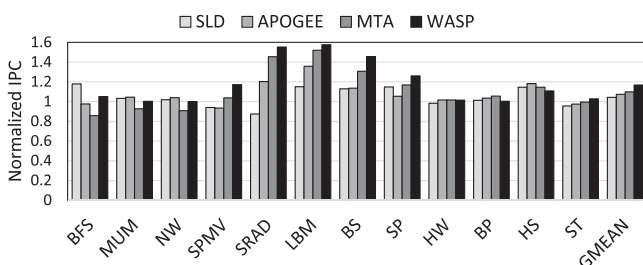


Fig. 10. Performance comparison between prefetching techniques.

Authorized licensed use limited to: Tsinghua University. Downloaded on March 06, 2026 at 08:53:43 UTC from IEEE Xplore. Restrictions apply.

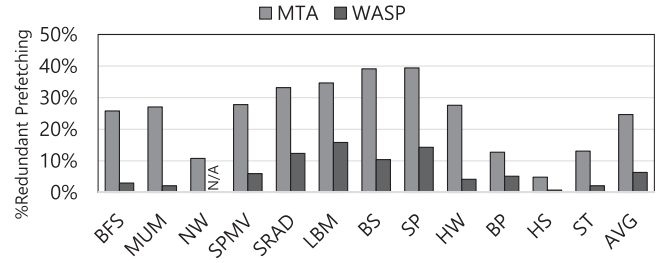


Fig. 11. Redundant prefetch ratio comparison with WASP and MTA.

All of the prefetching techniques mainly achieve high performance improvements in SRAD, LBM, BS, and SP, which are memory-intensive and have regular memory access patterns. They have the most appropriate behavioral characteristics for the prefetching on GPUs. Among the previously proposed techniques, MTA shows the best performance, with a performance improvement of 39.4 percent. By contrast, WASP achieves a 47.4 percent performance improvement in these applications.

In HW, BP, HS, and ST, none of the prefetching techniques achieve a significant speedup. This is because the memory pipeline stalls were not the main performance bottlenecks in these applications.

5.5 Prefetching Efficiency Comparison

We compare the ratio of redundant prefetches of WASP and MTA, which shows the best performances among the previously proposed techniques. Fig. 11 shows the experimental results. For all of the applications, WASP shows a 6.3 percent redundant prefetch ratio, while MTA shows 24.7 percent. For all types of applications, WASP significantly reduces the amount of redundant prefetches using the proposed prefetching target warps selection scheme.

In applications with irregular memory access patterns (BFS, MUM, NW, and SPMV), they execute a large portion of the loads in diverged branches. By preventing the generation of prefetch requests for these loads, WASP significantly reduces the redundant prefetches compared to MTA. In the case of NW, WASP generates no prefetch requests because all of the loads are executed in the diverged branches. On the other hand, MTA generates numerous prefetch requests in these applications. However, a large portion of the prefetch requests are redundant, as shown in Fig. 10. As a result, MTA shows a slowdown in these applications, while WASP prevents such a negative effect.

On the other hand, in SRAD, LBM, BS, and SP, reducing the redundant prefetches with WASP results in a significant performance improvement. In these applications, WASP shows a 13.2 percent redundant prefetch ratio, while MTA shows 36.6 percent. Reducing the redundant prefetches mitigates the harmful cache contentions. Therefore, WASP achieves a better speedup than MTA.

Moreover, WASP provides useful prefetches much more often than MTA. To show that, we compare the prefetch accuracies of WASP and MTA in Fig. 12. Overall, WASP shows a 79.7 percent prefetch accuracy, while MTA shows 62.9 percent. In SRAD, LBM,

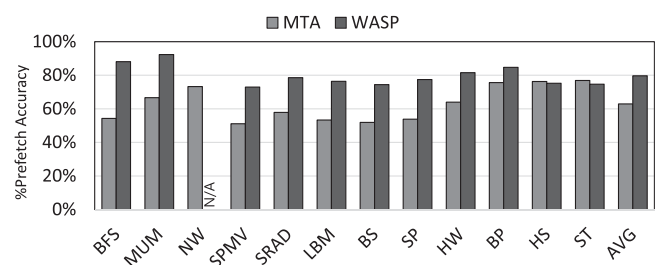


Fig. 12. Prefetch accuracy comparison.

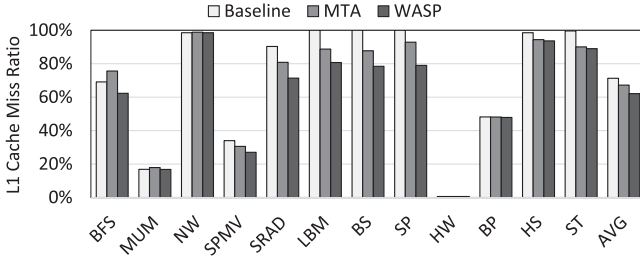


Fig. 13. L1 cache miss ratio comparison.

BS, and SP, WASP shows a 78.6 percent prefetch accuracy, which is significantly higher than the prefetch accuracy of MTA (57.9 percent in these applications). We observe that such a large gap in the prefetch accuracy results in a significant performance gap. With these results, we observe that the reduction in redundant prefetches results in an improvement in the prefetching efficiency. In addition, the accurate prefetching of WASP achieves a performance improvement.

5.6 Cache Miss Ratio and Memory Traffics Comparison

We analyzed how prefetching affected the GPU architectures. First, we compare the L1 data cache miss ratios of WASP, MTA, and the baseline GPU. Fig. 13 shows the experimental results. WASP reduces the L1 cache miss ratio by 9.0 percent on average, while MTA reduces it by 4.0 percent. In BFS, MUM, NW, and SPMV, MTA generates a large portion of harmful prefetches, as shown in Fig. 11. The harmful prefetches increase the cache contentions. Therefore, MTA shows an even worse cache miss ratio than the baseline in those applications. In the remaining applications, MTA reduces the cache miss ratio by 8.7 percent on average.

Excluding BFS, MUM, NW, and SPMV, WASP reduces the cache miss ratio by 12.1 percent on average. In SRAD, LBM, BS, and SP, WASP significantly reduces the L1 cache miss ratio as a result of the accurate prefetches. These operations result in a performance improvement of more than 20 percent in those applications.

If the prefetched data are re-accessed in the cache, both the cache miss ratio and off-chip memory traffics can be reduced. On the other hand, redundant prefetches may increase the memory traffics. The additional memory traffics caused by prefetching may be critical to the GPU performance.

We compare the off-chip memory traffics of WASP and MTA to the baseline in Fig. 14. On average, WASP and MTA reduce the memory traffics by 3.7 and 1.4 percent compared to the baseline, respectively. In all memory-intensive applications except NW, WASP reduces the memory traffics more than MTA. WASP achieves these results because of its more accurate prefetches, as shown in Fig. 12. Among the memory-intensive applications, a large portion of data is frequently re-accessed after being fetched from the off-chip memory in BFS, SPMV, and SRAD [9]. WASP accurately prefetches such data and presents a memory traffic reduction of more than 10 percent, as well as an increase in the cache hits in those applications.

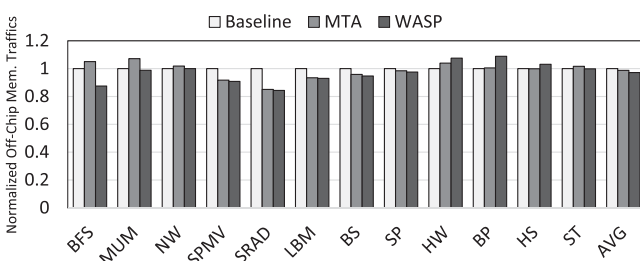


Fig. 14. Off-chip memory traffic comparison (Normalized to baseline).

Authorized licensed use limited to: Tsinghua University. Downloaded on March 06, 2026 at 08:53:43 UTC from IEEE Xplore. Restrictions apply.

In fact, WASP causes 7.5 percent (HW) and 8.0 percent (BP) more memory traffics compared to the baseline. We believe this is a result of the recursive prefetch operations for the early evicted data, which result in a large number of useful prefetches, as shown in Fig. 12.

6 CONCLUSION AND FUTURE WORK

Because GPUs execute hundreds of warps in an arbitrary order while executing an application, the prefetching technique for GPUs needs to provide accurate and timely prefetches. Therefore, we proposed a new prefetching technique called WASP. WASP includes a new technique to identify appropriate prefetching target warps. The main idea of the proposed technique is to monitor the dynamic load execution counts for all warps and utilize the monitoring results to select the prefetching target warps. With this technique, WASP achieves a better prefetching efficiency and speedup compared to the previously proposed techniques.

In future work, we plan to investigate the relationship between the redundant prefetches and the early evictions of prefetched lines, which is another issue for improving the prefetching efficiency. We will design a more improved data prefetcher considering this issue.

ACKNOWLEDGMENTS

This work was partly supported by Institute for Information & communications Technology Promotion (IITP) grant funded by the Korea government (MSIT) (No. R7117-16-0233, Development of Application Program Optimization Tools for High Performance Computing Systems) and Korea Evaluation Institute of Industrial Technology (KEIT) grant funded by the Korea government (MOTIE) (No.10080674, Development of Reconfigurable Artificial Neural Network Accelerator and Instruction Set Architecture).

REFERENCES

- [1] K. Li, X. Tang, and K. Li, "Energy-efficient stochastic task scheduling on heterogeneous computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 11, pp. 2867–2876, Nov. 2014.
- [2] O. Kayiran, A. Jog, M. Kandemir, and C. Das, "Neither more nor less: Optimizing thread-level parallelism for GPGPUs," in *Proc. 22nd Int. Conf. Parallel Archit. Compilation Techn.*, 2013, pp. 157–166.
- [3] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Cache-conscious wavefront scheduling," in *Proc. 45th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2012, pp. 72–83.
- [4] N. Vijaykumar, G. Pekhimenko, A. Jog, A. Bhowmick, R. Ausavarunirun, C. Das, M. Kandemir, T. C. Mowry, and O. Mutlu, "A case for core-assisted bottleneck acceleration in GPUs: Enabling flexible data compression with assist warps," in *Proc. ACM/IEEE 42nd Annu. Int. Symp. Comput. Archit.*, 2015, pp. 41–53.
- [5] G. Koo, Y. Oh, W. W. Ro, and M. Annamaram, "Access pattern-aware cache management for improving data utilization in GPU," in *Proc. 44th Annu. Int. Symp. Comput. Archit.*, 2017, pp. 307–319.
- [6] J. Lee, D. H. Woo, H. Kim, and M. Azimi, "GREEN cache: Exploiting the disciplined memory model of openCL on GPUs," *IEEE Trans. Comput.*, vol. 64, no. 11, pp. 3167–3180, Nov. 2015.
- [7] W. Jia, K. A. Shaw, and M. Martonosi, "Characterizing and improving the use of demand-fetched caches in GPUs," in *Proc. 26th ACM Int. Conf. Supercomputing*, 2012, pp. 15–24.
- [8] A. Sethia, G. Dasika, M. Samadi, and S. Mahlke, "APOGEE: Adaptive prefetching on GPUs for energy efficiency," in *Proc. 22nd Int. Conf. Parallel Archit. Compilation Techn.*, 2013, pp. 73–82.
- [9] Y. Oh, K. Kim, M. K. Yoon, J. H. Park, Y. Park, W. W. Ro, and M. Annamaram, "APRES: Improving cache efficiency by exploiting load characteristics on GPUs," in *Proc. 43rd Int. Symp. Comput. Archit.*, 2016, pp. 191–203.
- [10] C. J. Lee, O. Mutlu, V. Narasiman, and Y. N. Patt, "Prefetch-aware memory controllers," *IEEE Trans. Comput.*, vol. 60, no. 10, pp. 1406–1430, Oct. 2011.
- [11] T.-F. Chen and J.-L. Baer, "Effective hardware-based data prefetching for high-performance processors," *IEEE Trans. Comput.*, vol. 44, no. 5, pp. 609–623, May 1995.
- [12] V. Narasiman, M. Shebanow, C. J. Lee, R. Miftakhutdinov, O. Mutlu, and Y. N. Patt, "Improving GPU performance via large warps and two-level warp scheduling," in *Proc. 44th Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2011, pp. 308–317.

- [13] J. Lee, N. B. Lakshminarayana, H. Kim, and R. Vuduc, "Many-thread aware prefetching mechanisms for GPGPU applications," in *Proc. 43rd Annu. IEEE/ACM Int. Symp. Microarchitecture*, 2010, pp. 213–224.
- [14] A. Jog, O. Kayiran, A. K. Mishra, M. T. Kandemir, O. Mutlu, R. Iyer, and C. R. Das, "Orchestrated scheduling and prefetching for GPGPUs," in *Proc. 40th Annu. Int. Symp. Comput. Archit.*, 2013, pp. 332–343.
- [15] NVIDIA, CUDA C Programming Guide Version 9.0, Mar. 2018.
- [16] N. B. Lakshminarayana and H. Kim, "Spare register aware prefetching for graph algorithms on GPUs," in *Proc. IEEE 20th Int. Symp. High Perform. Comput. Archit.*, Feb. 2014, pp. 614–625.
- [17] C. Li, Y. Yang, H. Dai, S. Yan, F. Mueller, and H. Zhou, "Understanding the tradeoffs between software-managed vs. hardware-managed caches in GPUs," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, Mar. 2014, pp. 231–242.
- [18] D. Li, M. Rhu, D. R. Johnson, M. O'Connor, M. Erez, D. Burger, D. S. Fussell, and S. W. Keckler, "Priority-based cache allocation in throughput processors," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit.*, 2015, pp. 89–100.
- [19] S.-Y. Lee, A. Arunkumar, and C.-J. Wu, "CAWA: Coordinated warp scheduling and cache prioritization for critical warp acceleration of GPGPU workloads," in *Proc. 42nd Annu. Int. Symp. Comput. Archit.*, 2015, pp. 515–527.
- [20] A. Sethia, D. A. Jamshidi, and S. Mahlke, "Mascar: Speeding up GPU warps by reducing memory pitstops," in *Proc. IEEE 21st Int. Symp. High Perform. Comput. Archit.*, Feb. 2015, pp. 174–185.
- [21] S. Srinath, O. Mutlu, H. Kim, and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *Proc. IEEE 13th Int. Symp. High Perform. Comput. Archit.*, 2007, pp. 63–74.
- [22] W. W. L. Fung and T. M. Aamodt, "Thread block compaction for efficient SIMT control flow," in *Proc. IEEE 17th Int. Symp. High Perform. Comput. Archit.*, 2011, pp. 25–36.
- [23] S. Lee, K. Kim, G. Koo, H. Jeon, W. W. Ro, and M. Annavaram, "Warped-compression: Enabling power efficient GPUs through register compression," in *Proc. 42nd Annu. Int. Symp. Comput. Archit.*, 2015, pp. 502–514.
- [24] S. Che, M. Boyer, J. Meng, D. Tarjan, J. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *Proc. IEEE Int. Symp. Workload Characterization*, Oct. 2009, pp. 44–54.
- [25] J. A. Stratton, C. Rodrigues, I. J. Sung, N. Obeid, L. W. Chang, N. Anssari, G. D. Liu, and W. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," *Center Reliable High-Perform. Comput.*, 2012.
- [26] A. Bakhoda, G. Yuan, W. Fung, H. Wong, and T. Aamodt, "Analyzing CUDA workloads using a detailed GPU simulator," in *Proc. IEEE Int. Symp. Perform. Anal. Syst. Softw.*, 2009, pp. 163–174.
- [27] NVIDIA, NVIDIA CUDA SDK Code Sample 4.0, 2016.